Heapsort

Part II Sorting and Order Statistics

Scope and prerequisites

A binary heap represents a nearly complete binary tree by array indices, supporting extreme-key extraction and priority updates. Align parent/child indices, heap size, and every swap, prove linear bottom-up building, and distinguish sorting from priority-queue operations. Prerequisites are complete binary trees, array indexing, and sorting invariants.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

- Operate a max-priority queue using the max-heap property.
- Prove heapify costs and linear-time heap construction.
- Sort by repeatedly selecting the heap maximum and analyze O(n log n).

Sections 6.1–6.5 cover heaps, maintenance, building, heapsort, and priority queues.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.1 Heaps

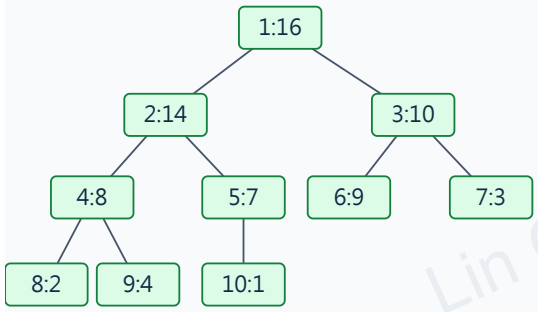
A binary heap is nearly complete: all but the last level are full, and the last fills left to right. In a 1-based array, root=1, parent(i)=floor(i/2), left(i)=2i, and right(i)=2i+1. Only indices at most heap-size exist.

A max-heap requires $A[\text{parent}(i)] \geq A[i]$ for every nonroot i. Repeated ancestor inequalities prove that the root is maximum. Siblings and the two subtrees have no mutual sorting order, so arbitrary-key binary search does not apply.

Array length and heap size differ. Heapsort excludes its fixed sorted suffix from heap size; heapify must not revisit it. For $n \geq 1$, depth h's leftmost index is 2^h, giving height $\text{floor}(\log_2 n)$.

6.1 Heaps (continued)

Ten-element max-heap: tree and array



Array indices and keys at those positions

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

Index rules

1-index：parent=floor(i/2)
Children: 2i,2i+1 if present
Heap size=10; all keys are active

Orange: current root. Green: valid heap roots.
Gray: pending. Each position index stays fixed.

6.2 Maintaining the heap property

MAX-HEAPIFY(A,i,s) assumes both child subtrees are already max-heaps, with only i possibly smaller than a child. Here s is heap size. An interface must reject i outside 1..s, or skip a call for an empty heap.

```
while true:
    largest=i
    if 2i<=s and A[2i]>A[largest]: largest=2i
    if 2i+1<=s and A[2i+1]>A[largest]: largest=2i+1
    if largest==i: return
    swap(A[i],A[largest])
    i=largest
```

Equivalently choose the larger child and compare it to the parent. Swapping merely because one child exceeds the other, without comparing the parent, is incorrect.

Heapify correctness and height cost

If the parent is largest, both parent-child edges and the already legal child subtrees form a heap. Otherwise the greatest child root is at least all its descendants. Moving it up makes both top edges legal. Only the old parent moved down can remain invalid.

Induct on remaining height to repair that subtree; the other subtree is unchanged. Each new i is 2i or 2i+1, one level lower, proving termination. A height-h subtree costs O(h+1) in the worst case; root repair costs O(log(n+1)). An already large parent may stop in O(1).

For A=[16,4,10,8,7,9,3],s=7,i=2, choose 8 over 7 and exchange 4↔8, obtaining [16,8,10,4,7,9,3]. At i=4, children 8 and 9 exceed s, so stop. The value 3 at index 7 is a child of index 3, not of 4.

6.3 Building a heap

```
s=length(A)
for i=floor(s/2) down to1:
    MAX-HEAPIFY(A,i,s)
```

Before an iteration, every index greater than i roots a legal heap. Initially floor(n/2)+1..n are leaves. Children of i have larger indices, so heapify's preconditions hold; afterward i also roots a legal heap. This maintains the next iteration. At i=0 the root is legal, so the whole array is a heap.

Empty and singleton arrays need no iterations. Index floor(n/2) need not be a leaf: with n=10, index 5 still has child 10 and their keys must be compared.

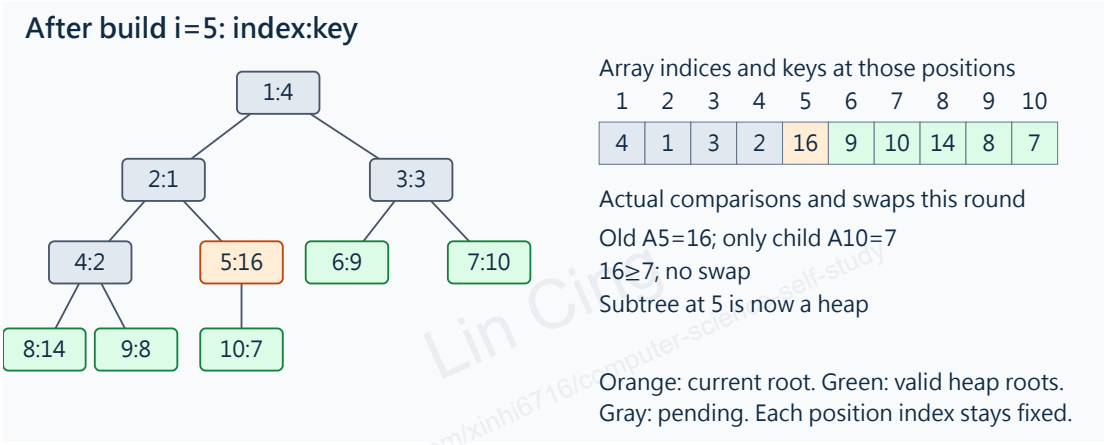
Five complete build rounds

Input A=[4,1,3,2,16,9,10,14,8,7],s=10.

i	Comparisons and swapped indices	Full result
5	16≥7 at child 10; no swap	[4,1,3,2,16,9,10,14,8,7]
4	choose 14 over 8; swap 4↔8	[4,1,3,14,16,9,10,2,8,7]
3	choose 10 over 9; swap 3↔7	[4,1,10,14,16,9,3,2,8,7]
2	choose 16 over 14; swap 2↔5, then 5↔10	[4,16,10,14,7,9,3,2,8,1]
1	choose 16, then 14, then 8; swap 1↔2,2↔4,4↔9	[16,14,10,8,7,9,3,2,4,1]

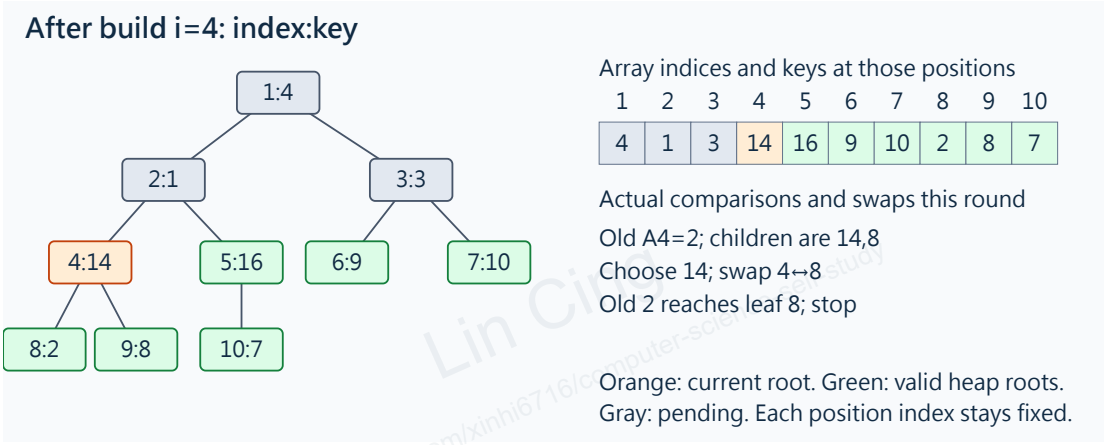
The next five figures use exactly these records. Orange marks the processed root, green roots already guaranteed to be heaps, and gray pending roots. Indices remain fixed during swaps; keys at those positions change.

Build at i=5: compare its only child



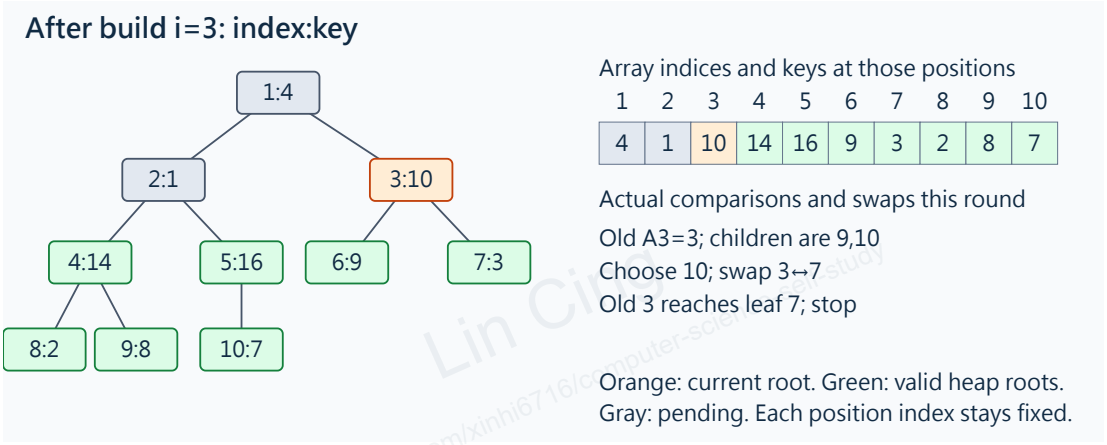
Value 16 at position 5 exceeds value 7 at 10, so no swap occurs. Positions 6..10 are leaves; the subtree rooted at 5 is now legal too.

Build at i=4: promote 14



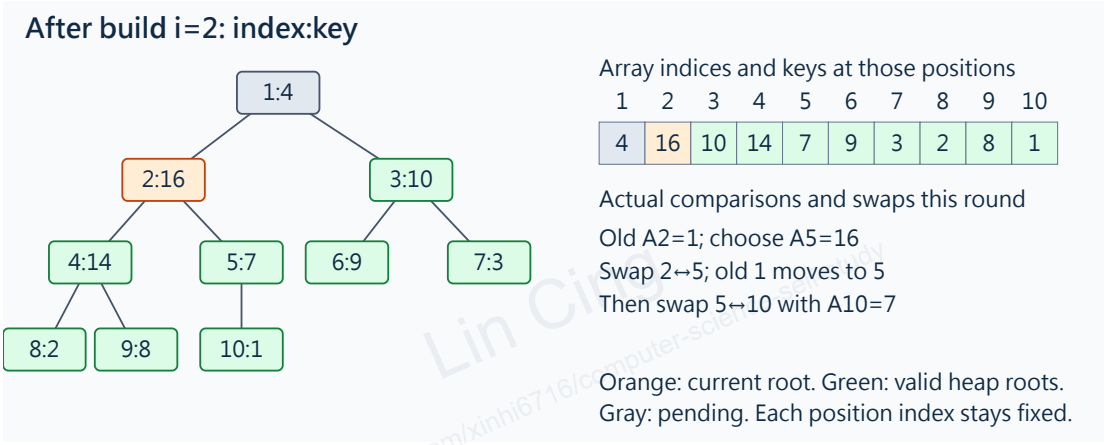
Original position 4 holds 2, with children 14 and 8. Choose 14 and exchange indices 4,8. Old 2 stops at leaf 8. Value 8 at index 9 is unchanged.

Build at i=3: promote 10



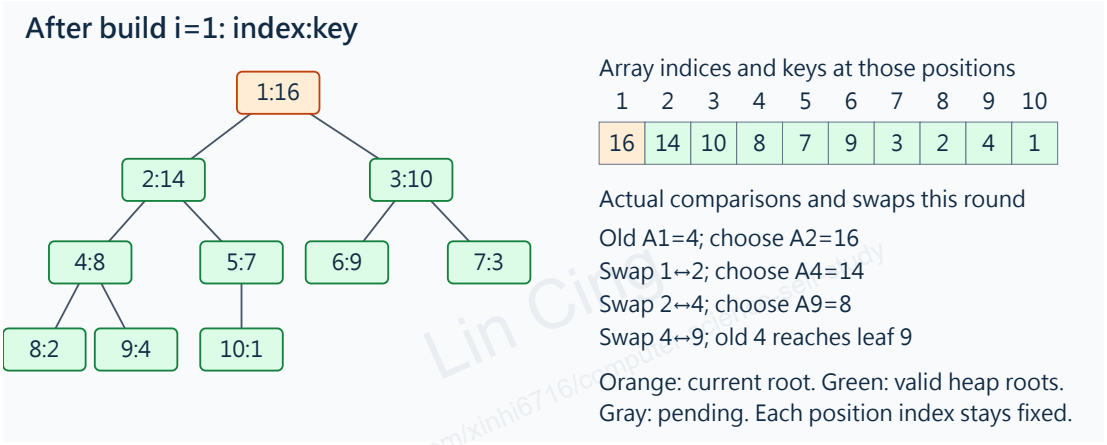
Original position 3 holds 3. Its children are 9 at index 6 and 10 at index 7. Swap 3,7; old 3 stops at leaf 7. Value 9 at 6 remains unchanged.

Build at i=2: descend through 5 to 10



Old 1 at position 2 first swaps with 16 at 5. It then has child 7 at 10, requiring another swap. Value 14 at index 4 is not moved by this round.

Build at i=1: descend through 2,4,9



Old root 4 exchanges with 16,14,8 and stops at index 9. Final edges give 16≥14,10; 14≥8,7; 10≥9,3; 8≥2,4; 7≥1. All ten keys are preserved.

Why heap construction is linear

Position i has height at least h iff its leftmost h-generation descendant $i \cdot 2^h \le n$. Exactly $\lfloor n/2^h \rfloor$ positions have height at least h. Count each node's height one level at a time:

$$\sum_i height(i) = \sum_{h \ge 1} \lfloor n/2^h \rfloor < n.$$

Each heapify costs O(height(i)+1), and there are floor(n/2) calls, proving O(n). For n≥2 the loop itself processes floor(n/2) roots, giving Ω(n), hence Θ(n). At n=15 the height sum is 7+3+1=11.

Repeated INSERT operations can instead have Θ(n log n) total upward travel. They are a different operation sequence and cannot use this bottom-up height sum.

6.4 The heapsort algorithm

```
BUILD-MAX-HEAP(A)
for end=length(A) down to2:
    swap(A[1],A[end])
    MAX-HEAPIFY(A,1,end-1)
```

Before each iteration, A[1..end] is a max-heap. The suffix A[end+1..n] is sorted increasingly and every suffix key is at least every heap key. Swapping fixes the current maximum at A[end]. Shrinking s and repairing only the prefix preserves the invariant. At end=1 all records are ordered. Swaps preserve their multiset.

Time is O(n log(n+1)); the comparison-sorting information bound gives worst-case Θ(n log n). With equal-key heapify stopping immediately, all-equal input can take Θ(n), so the worst bound does not apply equally to every input. Iterative heapify uses O(1) extra space; recursive heapify adds O(log n) stack.

6.4 The heapsort algorithm (continued)

For the lower bound, distinct keys have n! possible input orders. A correct comparison decision tree needs at least n! leaves; a binary tree of depth d has at most 2^d leaves. Thus $d \geq \log_2(n!)$. The final floor(n/2) product factors are each at least n/2, giving $\log_2(n!) \geq \text{floor}(n/2)\log_2(n/2) = \Omega(n \log n)$. At least one input needs that many comparisons.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Every ten-element heapsort round

Each round swaps the root with the last heap position, then repairs the remaining prefix.

New heap size	Repaired heap	Sorted suffix
9	[14,8,10,4,7,9,3,2,1]	[16]
8	[10,8,9,4,7,1,3,2]	[14,16]
7	[9,8,3,4,7,1,2]	[10,14,16]
6	[8,7,3,4,2,1]	[9,10,14,16]
5	[7,4,3,1,2]	[8,9,10,14,16]
4	[4,2,3,1]	[7,8,9,10,14,16]
3	[3,2,1]	[4,7,8,9,10,14,16]
2	[2,1]	[3,4,7,8,9,10,14,16]
1	[1]	[2,3,4,7,8,9,10,14,16]

Every ten-element heapsort round (continued)

In round two, replacement root 1 follows only indices 1→3→6, exchanging with 10 and 9, not with 8 or 2. A swap path is not a list of all values in the final array.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.5 Priority queues

MAXIMUM reads A[1] in O(1). EXTRACT-MAX saves it, replaces it with the final key, decreases s, and heapifies, taking worst-case O(log(n+1)). Empty input returns EMPTY instead of reading A[1].

INCREASE-KEY(A,i,k) requires k at least the old key. Change it, then swap upward while the parent is smaller. Stop at a sufficient parent or the root. Only ancestor edges can fail; each upward step repairs one, giving correctness and worst-case O(log n).

INSERT appends a position with a formal negative-infinity key, then increases it. Equivalently write the new key directly and use the same upward repair without a representable infinity. Dynamic-array growth needs separate amortized analysis; fixed capacity first checks FULL.

For A=[20,15,18,7,12,9,17,2], increase position 8 to 16. Swap 8↔4 to obtain [20,15,18,16,12,9,17,7], then 4↔2 to obtain [20,16,18,15,12,9,17,7]. Parent 20 stops the ascent.

Decreases, deletion, and stability

A max-heap key decrease can violate only child edges, so repair downward; an increase repairs upward. To delete position i, fill from the final position and decrease s. If the replacement exceeds its parent, move upward; otherwise heapify downward. Deleting the last position needs only shortening. Each repair costs O(log n), but searching for an arbitrary key to obtain i can still cost Θ(n).

If clients identify records by handles, every swap must update the handle-to-index map. An array index does not permanently identify one record.

Heapsort is generally unstable. With (2,a),(2,b),(1,c) and no equal-key heapify swap, output can be (1,c),(2,b),(2,a), reversing a,b. Adding original positions to comparison keys can enforce stability but changes comparison and storage requirements.

Sources

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 6.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study